

Another Cycle Command

Summary: Phase 2: `/explore` Deep codebase exploration Structure, architecture, patterns, dependencies Key functionality mapping Integration points identification.

Generated: 2026-01-12 23:47 UTC | Ideas Extracted: 56

What Happened

Phase 2: `/explore` Deep codebase exploration Structure, architecture, patterns, dependencies Key functionality mapping Integration points identification.

Key Difference: `/another-cycle` is the complete lifecycle, `/run-it` is a focused systematic workflow.

Use when: Starting major new work, need complete systematic approach, want full cycle from onboarding through quality assurance and evolution.

The workflow covers the entire development lifecycle: Orientation: Get oriented and understand the codebase Analysis: Validate assumptions, form hypotheses, critique, orchestrate Engineering: Plan and implement systematically Quality: Improve, verify, and prove quality Evolution: Track Being evolution and genetic lineage Planning: Identify next steps and track goals.

Phase 8: `/engineer` Complete engineering workflow Spin-up → explore → draft plan → critique plan → finalize plan → begin Systematic implementation approach Active tool usage (Empirica, `_pyrite`, GitHub MCP, work-efforts).

Group 6 (Planning): Next: Next step recommendation Goal: Goal tracking updates Checkpoint: Checkpoint file in `workefforts/`.

Use `/another-cycle` when: ☒ Starting major new initiative ☒ Need complete systematic approach ☒ Want full cycle from start to finish ☒ Need comprehensive quality assurance ☒ Want Being evolution tracking ☒ Need complete lifecycle coverage ☒ Starting significant new work.

Orientation Complete: Onboarded and explored codebase Analysis Complete: Assumptions validated, hypotheses formed, critique done, orchestrated, analyzed Engineering Complete: Systematic implementation done Quality Complete: Improvements identified, version-baked, run-it executed, proven Evolution Complete: Being evolved, learnings captured Planning Complete: Next steps identified, goals tracked, checkpoint created.

This command provides the complete development lifecycle from orientation through evolution - perfect for major initiatives requiring comprehensive systematic approach.

Phase 5: `/critique` Security-first adversarial review Find vulnerabilities, assumptions, overengineering Prioritize by severity (CRITICAL first) Generate critique report.

Continue if Possible: Skip failed phase if not critical Continue with remaining phases Note what was skipped.

Graceful Degradation: The command continues with available phases even if some fail. Critical phases may cause subsequent phases to skip, but non-critical failures shouldn't stop the entire cycle.

Key Difference: `/another-cycle` includes engineering as one phase, `/engineer` is the full engineering workflow.

Runs essential phases only. Use when: Time-constrained Need faster execution Some phases not applicable.

This command provides: Complete Orientation: Full onboarding and exploration Comprehensive Analysis: Assumption validation, hypothesis formation, critique, orchestration Full Engineering: Complete engineering workflow Quality Assurance: Improvements, version-bake, run-it, proof Evolution: Being evolution from Source Reflection: Journal and learning capture Planning: Next steps, goals, checkpoints.

Each phase builds on the previous: Orientation provides foundation for analysis Analysis informs engineering decisions Engineering creates work for quality assurance Quality ensures evolution is meaningful Evolution captures learnings for future cycles.

Every phase produces evidence: Assumptions validated with traces Hypotheses verified with evidence Decisions supported by data All findings documented and traceable.

Phase 1: `/onboard` Standard repository onboarding Proceed → spin-up → analyze → phase1 → prepare → recap Get oriented to the project Initial analysis and setup.

Phase 6: `/comprehensive-orchestration` Full orchestration workflow (prompt template) Spin-up → consider → engineer → visualize → analyze → goal → checkpoint → execute → hypothesis → verify → reflect → recap → proceed → decide Complete strategic planning Note: This is a prompt template, not an executable command - execute phases manually or use individual commands.

Phase 7: `/analyze` Data analysis, insights, and action planning Health analysis Issue identification Opportunity discovery Pattern analysis Prioritized action plan.

Phase 9: `/improve` Identify and prioritize improvements Code quality, documentation, architecture, testing, performance, usability Score-based ranking Actionable recommendations.

Phase 10: `/version-bake` Complete quality workflow Reflect → run-it → improve → check-assumptions → verify → hypothesis → prove-it Tracks genetic lineage of ideas Self-correcting loop.

Phase 12: `/prove-it` Prove scientific method tool works Run simple and real experiment proofs
Show state capture (A & B) and data collection (C) Verify file persistence.

Phase 14: `/journal` AI journal system hub View recent entries, search, statistics Write new reflections
Capture learnings.

Phase 15: `/next` Identify next step based on goals Analyze active goals, work in progress, context
Prioritize by importance and urgency Provide actionable recommendation.

Phase 16: `/goal` Track larger goals and break into steps Review current goals Update progress Create
new goals if needed.

Phase 17: `/checkpoint` Create situation report and status update Recap conversation Document
current state Update devlog and work efforts.

Group 4: Quality Assurance `/improve` → Improvement analysis `/version-bake` → Quality workflow `/run-it`
→ Comprehensive workflow `/prove-it` → Scientific method proof.

Additional Details

Group 6: Planning & Tracking /next → Next step identification /goal → Goal tracking /checkpoint → Status checkpoint.

Actions: Check current state (date, directory, git status) Verify prerequisites (tools, dependencies) Identify current work effort (if applicable) Create cycle tracking document Display cycle overview.

Actions: Execute /onboard - Standard onboarding Execute /explore - Deep exploration Document findings from orientation Update cycle tracking.

Actions: Execute /check-assumptions - Validate assumptions Execute /hypothesis - Form hypotheses Execute /critique - Adversarial review Execute /comprehensive-orchestration - Full orchestration Execute /analyze - Data analysis Document all analysis findings Update cycle tracking.

Actions: Execute /engineer - Complete engineering workflow Document implementation progress Update cycle tracking.

Actions: Execute /improve - Improvement analysis Execute /version-bake - Quality workflow Execute /run-it - Comprehensive workflow Execute /prove-it - Scientific method proof Document quality findings Update cycle tracking.

Actions: Execute /evolve - Being evolution Execute /journal - Journal hub Document evolution and reflections Update cycle tracking.

Actions: Execute /next - Next step identification Execute /goal - Goal tracking Execute /checkpoint - Status checkpoint Document final state Complete cycle tracking.

Document the Failure: Note which phase failed Document error message Capture context at time of failure.

Suggest Remediation: Provide steps to fix failure Suggest alternative approaches Recommend when to retry.

Group 4 (Quality): Improve: Improvement report Version-Bake: Quality workflow documents Run-It: Multiple workflow documents Prove-It: Proof results in `scientificmethodtool/experiments/`.

This command orchestrates: `/onboard` - Standard onboarding `/explore` - Deep exploration `/check-assumptions` - Assumption validation `/hypothesis` - Hypothesis formation `/critique` - Adversarial review `/comprehensive-orchestration` - Full orchestration `/analyze` - Data analysis `/engineer` - Engineering workflow `/improve` - Improvement analysis `/version-bake` - Quality workflow `/run-it` - Comprehensive workflow `/prove-it` - Scientific method proof `/evolve` - Being evolution `/journal` - Journal hub `/next` - Next step identification `/goal` - Goal tracking `/checkpoint` - Status checkpoint.

`/another-cycle` includes: Onboard and explore (orientation) Comprehensive-orchestration (full strategic planning) Analyze (data analysis) Engineering (implementation workflow) Improve (improvement analysis) Version-bake (quality workflow) Evolve (Being evolution) Journal (reflection hub) Next, goal, checkpoint (planning).

`/run-it` focuses on: Systematic workflow with verification and critique 15 phases of analysis, verification, decision-making.

`/engineer` focuses on: Engineering workflow (spin-up → explore → draft → critique → finalize → begin).

Don't use `/another-cycle` when: ❌ Quick task or simple change ❌ Already have full understanding ❌ Time-constrained (use `--quick` or individual commands) ❌ Just need one specific workflow ❌ Routine maintenance work.

Executes all 17 phases in 6 groups. Use when: Starting major new initiative Need complete lifecycle coverage Want comprehensive quality assurance.

Runs only quality assurance group. Use when: Need focused quality workflow Other areas already complete Want targeted analysis.

Runs cycle without evolution and journal. Use when: Don't need Being evolution Want to skip reflection phases Customizing workflow.

Runs only specified phases. Use when: Want custom workflow Need specific phases only Building custom sequence.

The command requires: All individual commands to be available Empirica CLI (for some phases) MCP servers (for some phases) Git repository (for some checks) Scientific method tool (for prove-it).

If dependencies are missing: Document which phases are skipped Continue with available phases Provide guidance on installing missing dependencies.

Group 3: Engineering & Implementation /engineer → Complete engineering workflow.

Purpose: Systematic engineering and implementation.

Group 3 (Engineering): Engineering: Engineering documents, plans, implementation.

Use for Major Initiatives: This is a comprehensive cycle - use for major work, not quick tasks.

Trust the Process: The systematic progression is intentional - trust it.

Document Everything: All phases create documentation - use it for future reference.

Content Breakdown

Type	Count
Decisions	6
Insights	2
Actions	22
Concepts	19
Questions	7